The knapsack problem

A traveler has a backpack with a maximum capacity of **W** (weight units). There are **n** available items, each with a weight p_i and a benefit value b_i. The goal is to select a subset of these items such that the **total benefit** inside the backpack is **maximized**, without exceeding the capacity **W**.

Input:

- **n**: Total number of available items.
- **W**: Maximum backpack capacity in weight units.
- A list of **n** items, each with:
  - $\mathbf{p_i}$: Weight of item i.
  - $\mathbf{b_i}$: Benefit of item i.

Expected Output:

- A subset of items such that:
  - The total weight **does not exceed W**.
  - The total benefit is **maximized**.

Example Input Data:

Let's assume we have the following items to be put in a backpack with capacity: **W = 3000 gr**:

	Item	Weight	Benefit
1	Gloves	150	3
2	Rain coat	100	5
3	Down Jacket	400	7
4	Headlamp	180	4
5	Poles	500	8
6	Canteen	1500	10
7	Spare batteries	500	8
8	Solar charger	450	9
9	GPS	380	10
10	Sandwich	250	10
11	Extra food	250	7
12	Note book, pencil	80	2
13	Lip balm	20	3
14	Sunscreen	60	8
15	Insect repellent	45	5
16	Basic first aid kit	200	9

Solution Using a Genetic Algorithm

To solve this problem with a **genetic algorithm**, follow these steps:

1. **Solution Encoding**: Choose a suitable representation to this problem.
2. **Fitness Function**: Compute the total value of the selected items, penalizing solutions that exceed the maximum weight.
3. **Selection**: Use a selection operator such as tournament to choose promising solutions.
4. **Crossover**: Combine two parent solutions to generate new ones.
5. **Mutation**: Implement a mutation operator suitable for the chosen representation.
6. **Stopping Criteria**: Determine the appropriate conditions to stop the optimization procedure.